ÉTUDE INFORMATIQUE

Système automatique de tri par couleur

Logique du programme, algorithmes et choix de conception
du microprogramme de l'ESP32 maître

Club d'Informatique Industrielle · IGE × EPAC

Encadrant : Kevin HOUNSA

3 novembre 2026

Sommaire

Liste des symboles 2

Liste des tableaux 3

Liste des figures 4

Partie 1 : Introduction 5

1.1 But de l'étude 5

1.2 Principe de fonctionnement du point de vue logiciel 5

1.3 Architecture générale 6

1.4 Organisation du code source 6

Partie 2 : Étude des concepts de base 7

2.1 Gérer le temps sans bloquer 7

2.2 Machine à états et drapeaux 8

2.3 Lire un capteur : seuil, front et anti-rebond 9

2.4 La file FIFO circulaire 10

2.5 Générer des signaux avec le PWM matériel 10

2.6 La liaison série UART 11

2.7 Le bus I2C et l'écran LCD 12

2.8 Multitâche : deux cœurs, une tâche, une file de messages 12

2.9 Mémoire non volatile : `Preferences` 13

2.10 Supervision à distance avec Blynk 13

2.11 Heure réseau et journal d'événements 14

Partie 3 : Modules du programme 14

3.1 Module 1 : Configuration et paramètres 14

3.2 Module 2 : Acquisition des capteurs laser 15

3.3 Module 3 : File des cubes et attribution des couleurs 17

3.4 Module 4 : Réception des ordres de la caméra 18

3.5 Module 5 : Commande du moteur du tapis 19

3.6 Module 6 : Orientation du bac par le servomoteur 20

3.7 Module 7 : Sécurités 22

3.8 Module 8 : Signalisation par LED et écran LCD 23

3.9 Module 9 : Supervision par le cloud Blynk 25

3.10 Module 10 : Comptage, mémoire permanente et remise à zéro 26

Partie 4 : Intégration 27

4.1 Schéma bloc logiciel 27

4.2 Initialisation et boucle principale 27

4.3 Tableau des broches du microcontrôleur 30

4.4 Tableau des broches virtuelles Blynk 30

4.5 Scénario complet : le passage d'un cube vert 30

4.6 Plan de test 30

4.7 Limites et perspectives 32

Partie 5 : Conclusion 32

Bibliographie 34

Annexe A : Index des fonctions 35

LISTE DES SYMBOLES

N_{rev}	: nombre de pas par tour du moteur (200 pas pour 1,8°)
μ	: facteur de micro-pas réglé sur le driver (1 dans ce projet)
n (`rpm`)	: vitesse de rotation du rouleau d'entraînement (tr/min)
f_{pas}	: fréquence des impulsions envoyées sur l'entrée PUL du driver (Hz)
P	: périmètre du rouleau d'entraînement du tapis (cm)
v	: vitesse linéaire du tapis (cm/s)
D	: distance entre le capteur de fin et le bac, qui est aussi l'espacement minimal entre deux cubes (cm)
T_{chute}	: délai de chute, temps nécessaire à un cube pour parcourir D (ms)
S	: seuil de détection appliqué à la valeur du convertisseur analogique-numérique
x_{ADC}	: valeur lue sur une entrée analogique, de 0 à 4095 (12 bits)
Δt_{deb}	: durée d'anti-rebond d'un capteur (ms)
t_{now}, t_0	: instant courant et instant mémorisé, mesurés par `millis()` (ms)
T_{pause}	: durée d'un arrêt d'urgence (ms)
T_{PWM}	: période du signal PWM du servomoteur (20 ms)
t_{imp}	: largeur d'impulsion du signal du servomoteur (μs)
R	: résolution du signal PWM, en bits (14 bits pour le servomoteur)
N_{max}	: capacité de la file de cubes (10)
f_{msg}	: cadence maximale d'envoi de messages vers le cloud (messages/s)
ADC	: convertisseur analogique-numérique (*Analog-to-Digital Converter*)
FIFO	: file « premier entré, premier sorti » (*First In, First Out*)
GPIO	: broche d'entrée-sortie à usage général
HSV	: espace colorimétrique teinte-saturation-valeur, utilisé par la caméra
I2C	: bus série à deux fils (SDA, SCL), utilisé pour l'écran LCD
LCD	: écran à cristaux liquides, 16 colonnes sur 2 lignes
LEDC	: périphérique PWM matériel de l'ESP32
NTP	: protocole de synchronisation de l'heure par le réseau
NVS	: mémoire non volatile de l'ESP32 (accès par la bibliothèque `Preferences`)
PUL, DIR, ENA	: entrées d'impulsion, de sens et d'activation du driver TB6600
PWM	: modulation de largeur d'impulsion
RTOS	: système d'exploitation temps réel (ici FreeRTOS)
UART	: liaison série asynchrone entre la caméra et l'ESP32
V-pin	: broche virtuelle Blynk (V0, V1, …), canal d'échange avec le tableau de bord
8N1	: trame série à 8 bits de données, sans parité, avec 1 bit d'arrêt
J, V, B, X	: codes couleur échangés avec la caméra : Jaune, Vert, Bleu, inconnu
A	: code d'arrêt d'urgence échangé avec la caméra
`?`	: couleur non encore attribuée à un cube de la file

Liste des tableaux

1 Déroulé d'un cube et traitement logiciel associé. 5
2 Organisation du fichier source et module correspondant. 7
3 États du système et drapeaux qui les définissent. 8
4 États d'un capteur laser et interprétation logicielle. 9
5 Protocole de la liaison caméra → ESP32. 12
6 Les deux sens d'échange avec Blynk. 14
7 Paramètres de réglage du programme. 15
8 Champs de la structure `Cube`. 17
9 Effet de la vitesse sur la fréquence de pas, la vitesse du tapis et le délai de chute. 20
10 Positions du bac : angle, largeur d'impulsion et rapport cyclique codé sur 14 bits. 21
11 Les quatre sécurités du système. 22
12 Commande des LED selon l'état du système. 24
13 Messages affichés sur l'écran LCD (16 caractères par ligne). 24
14 Ordre des étapes de la boucle principale et justification. 28
15 Affectation des broches de l'ESP32 maître. 29
16 Broches virtuelles du tableau de bord et règles de traitement. 30
17 Déroulé d'un cube vert, de la détection au comptage. 31
18 Programmes de test et facilités de débogage. 31
19 Fonctions du programme, module de rattachement et rôle. 35

Liste des figures

1 Schéma des communications : qui parle à qui, et par quel moyen. 6
2 Diagramme des états et des transitions. 9
3 Détection de front avec anti-rebond : un rebond à l'arrivée du cube est rejeté. . . . 10
4 File circulaire de $N_{\max} = 10$ cases contenant cinq cubes : `cubesOnBelt` $= 5$. Les cases 7, 8 et 9 puis 0 et 1 sont occupées ; `?` désigne une couleur non encore reçue. 11
5 Répartition du travail entre les deux cœurs et moyens de communication. 13
6 Algorithme de `risingEdge()` : détection d'un front avec anti-rebond. 16
7 Cycle de vie d'un cube dans la file. L'orientation du bac a lieu à l'arrivée si le cube est en tête de file, sinon dès que le cube précédent est trié. 18
8 Aiguillage d'un octet reçu. 19
9 Orientation différée du bac pour deux cubes proches : le bac change seulement quand A a fini de tomber. 21
10 Gel des chronomètres : le délai de chute ne compte pas le temps passé en pause. . 23
11 Logique de remise à zéro des compteurs. 26
12 Schéma bloc logiciel : entrées, état central, sorties. 27
13 Organisation de la boucle principale. 29

PARTIE 1 : INTRODUCTION

1.1 But de l'étude

L'étude électrique a fixé le matériel : capteurs, driver, moteur, servomoteur, écran. Ce document traite l'autre moitié du projet, le *programme* qui fait vivre ce matériel. Il ne se contente pas de décrire le code ligne par ligne : il explique les *logiques* utilisées pour l'écrire, c'est-à-dire les problèmes rencontrés (attendre sans bloquer, ignorer les rebonds, suivre plusieurs cubes à la fois, rester réactif malgré un Wi-Fi capricieux, s'arrêter en sécurité) et la solution retenue pour chacun.

La démarche suit celle de l'étude électrique. La partie 2 présente les notions de base nécessaires. La partie 3 découpe le programme en dix modules, chacun présenté selon le même plan : rôle, outils utilisés, schéma, fonctionnement et dimensionnement. La partie 4 montre comment ces modules sont assemblés dans la boucle principale, puis donne les tables de référence (broches, broches virtuelles, plan de test).

1.2 Principe de fonctionnement du point de vue logiciel

Pour le programme, le système est un enchaînement de six étapes (figure 1 pour les liaisons, tableau 1 pour le déroulé).

Table 1. Déroulé d'un cube et traitement logiciel associé.

N°	Étape	Événement	Traitement
1	Détection	Front du laser de début	Création d'un cube dans la file, démarrage différé du tapis
2	Identification	Octet reçu de la caméra (J, V, B ou X)	Attribution de la couleur au plus ancien cube sans couleur
3	Transport	Écoulement du temps	Le tapis tourne à vitesse constante ; chaque cube garde son état
4	Fin de tapis	Front du laser de fin	Le cube est marqué « arrivé » ; le bac est orienté si le cube est en tête
5	Chute	Délai T_{chute} écoulé	Le cube est considéré comme trié et retiré de la file
6	Comptage	Fin du tri	Compteurs, mémoire permanente, tableau de bord

Le programme est *piloté par les événements* : il ne déroule pas un scénario fixe, il réagit à ce qui arrive (un front sur un capteur, un octet sur la liaison série, une commande du tableau de bord, un délai écoulé) en mettant à jour un état, puis en pilotant les sorties à partir de cet état. Trois idées directrices structurent tout le code :

— **ne jamais bloquer** : aucun `delay()` dans la logique de tri ; les attentes sont des comparaisons de temps, et le réseau tourne dans une tâche séparée ;

— **un état unique, plusieurs vues** : les LED, l'écran et le tableau de bord sont tous calculés à partir des mêmes variables d'état, ils ne peuvent donc pas se contredire ;
— **la sécurité d'abord** : dans la boucle principale, l'arrêt d'urgence est examiné avant toute autre action.

1.3 Architecture générale

Figure 1 montre qui parle à qui. L'ESP32 est le maître : il lit les capteurs, commande le tapis et le bac, affiche l'état et dialogue avec le cloud. La carte XIAO ESP32-S3 est dédiée à la vision : à partir de l'image de la caméra et de la mesure de l'ultrason, elle détermine la couleur du cube (par la teinte, en espace HSV) et l'envoie à l'ESP32 sur la liaison UART.

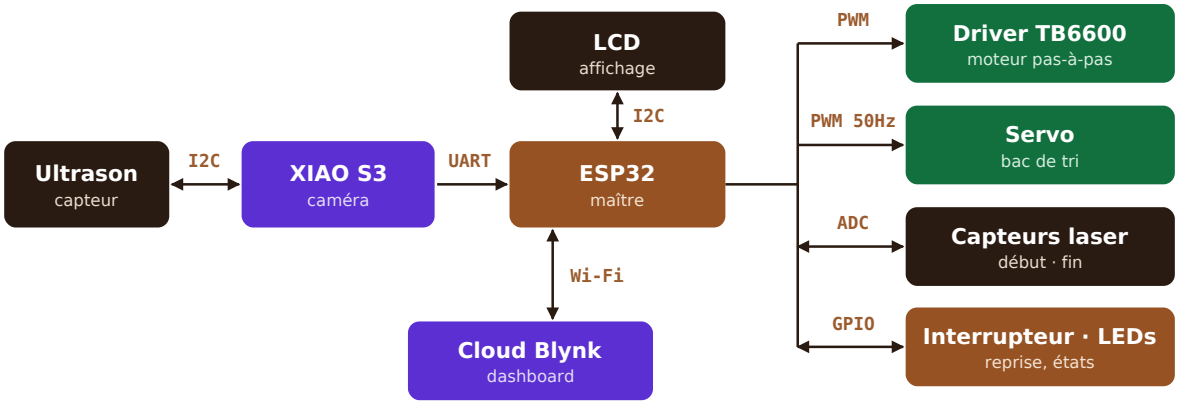


FIGURE 1. Schéma des communications : qui parle à qui, et par quel moyen.

Chaque liaison correspond à un moyen de communication adapté à son rôle :

— **UART** (caméra → ESP32) : un octet par message, à 9600 bauds ; simple et peu coûteux ;
— **PWM** (ESP32 → driver et servo) : le signal est produit par le périphérique matériel, sans occuper le processeur ;
— **ADC** (capteurs laser → ESP32) : la valeur analogique est lue et comparée à un seuil par le programme ;
— **I2C** (ESP32 → LCD) : deux fils seulement pour l'écran ;
— **GPIO** : interrupteur de reprise et LED d'état ;
— **Wi-Fi** (ESP32 ↔ cloud Blynk) : échange bidirectionnel avec le tableau de bord.

> **Périmètre.** Cette étude porte sur le microprogramme de l'ESP32 maître. La partie vision (acquisition d'image, seuils HSV, ultrason) s'exécute sur la carte XIAO et fait l'objet d'un document distinct ; elle n'apparaît ici que par son interface, la liaison UART décrite au section 2.6. L'étage électrique est décrit dans l'étude du système électrique.

1.4 Organisation du code source

Le programme tient dans un seul fichier, découpé en sections numérotées (tableau 2). Cet ordre n'est pas arbitraire : les constantes précèdent les structures qui les utilisent, et les fonctions utilitaires précèdent celles qui les appellent, car le compilateur lit le fichier de haut en bas.

TABLE 2. Organisation du fichier source et module correspondant.

Section	Contenu	Module (partie 3)
0	Identifiants Blynk (avant les `##include`)	1 Configuration
1, 2	Brochage et paramètres de réglage	1 Configuration
3	État du système, structure `Cube`, file des cubes	3 File des cubes
3bis	Tâche Blynk, messages et drapeaux	9 Supervision Blynk
4	Gestion du temps	Partie 2
5	Moteur, servomoteur, LCD, nom des couleurs	5, 6 et 8
5bis	Compteurs, mémoire, état envoyé au tableau de bord	9 et 10
6	Capteurs laser (front, anti-rebond)	2 Capteurs
7	LED d'état et d'alerte	8 Signalisation
8	Arrêt d'urgence et reprise	7 Sécurités
8bis	Commandes venant du tableau de bord	9 Supervision Blynk
9	`setup()` : initialisation	4.2
10	Logique : cubes, couleurs, fin de tapis, erreur	3, 4 et 7
11	`loop()` : boucle principale	4.2

PARTIE 2 : ÉTUDE DES CONCEPTS DE BASE

Cette partie présente, indépendamment du projet, les notions de programmation embarquée sur lesquelles repose le code. Chaque notion se termine par un paragraphe « Dans ce projet » qui indique où et pourquoi elle est utilisée.

2.1 Gérer le temps sans bloquer

2.1.1 Le problème de `delay()`

Un programme qui appelle `delay(1000)` ne fait rien d'autre pendant une seconde : il ne lit plus ses capteurs, ne reçoit plus la caméra et ne réagit plus à l'arrêt d'urgence. Sur un tapis en mouvement, un cube peut franchir un capteur pendant ce temps sans être vu.

2.1.2 La solution : comparer des instants

La fonction `millis()` renvoie le nombre de millisecondes écoulées depuis le démarrage, sur 32 bits non signés. Au lieu d'attendre, on mémorise l'instant t_0 où un événement a eu lieu, puis on teste à chaque passage de boucle si la durée voulue s'est écoulée :

$$\Delta t = (t_{\text{now}} - t_0) \bmod 2^{32}. \tag{1}$$

Le programme continue de tourner entre deux tests ; l'attente est devenue une simple comparaison.

```
unsigned long elapsedSince(unsigned long t) { return millis() - t; }

if (elapsedSince(lastHeartbeat) < HEARTBEAT_MS) return;   // not yet: carry on
```

Le compteur revient à zéro après 2^{32} ms, soit environ 49,7 jours. La soustraction de deux entiers non signés de 32 bits se fait modulo 2^{32} : le résultat reste juste même si `millis()` a débordé entre-temps, à condition que l'intervalle mesuré soit inférieur à 49,7 jours. Exemple : $t_0 = 4\,294\,967\,000$ et $t_{\text{now}} = 200$ après débordement donnent $\Delta t = 200 - 4\,294\,967\,000 + 2^{32} = 496$ ms, valeur correcte.

> **Dans ce projet.** Ce motif sert partout : démarrage différé du tapis, anti-rebond des capteurs, clignotement des LED, rafraîchissement de l'écran, ligne d'état périodique, chronomètres des cubes. La fonction `elapsedSince()` relit `millis()` à chaque appel, plutôt que de réutiliser une valeur lue en début de boucle : une écriture sur l'écran ou en mémoire flash peut durer assez longtemps pour fausser une valeur mémorisée.

2.2 Machine à états et drapeaux

Un système automatique se comporte différemment selon son état : il ne réagit pas de la même façon à un capteur quand il est en marche ou en arrêt d'urgence. Le moyen le plus lisible de raisonner est la machine à états : un ensemble d'états, et des transitions déclenchées par des événements.

Le programme n'a pas de variable « état » unique. Il combine quatre drapeaux booléens (tableau 3), ce qui évite d'énumérer toutes les transitions : chaque drapeau est modifié à un seul endroit du code.

Table 3. États du système et drapeaux qui les définissent.

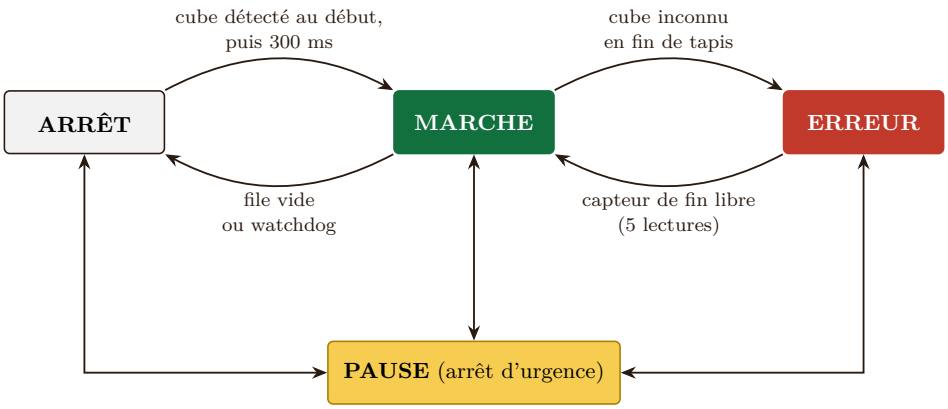
État	Condition	Signification
PAUSE	`paused` vrai	Arrêt d'urgence : moteur coupé, tout est ignoré sauf la reprise
ERREUR	`cubeError` vrai, `paused` faux	Un cube inconnu bloque le capteur de fin ; attente du retrait manuel
MARCHE	`beltActive` vrai, les deux autres faux	Le tapis tourne et les cubes sont triés
ARRÊT	aucun des cas précédents	Tapis à l'arrêt, en attente d'un cube

L'ordre du tableau est celui des priorités : `paused` l'emporte sur `cubeError`, qui l'emporte sur `beltActive`. Le drapeau `beltStartPending` n'est pas un état mais une étape transitoire : un cube vient d'être détecté et le démarrage du tapis est programmé après un court délai. La figure 2 résume les transitions.

> **Dans ce projet.** Les LED, l'écran et le tableau de bord se calculent tous à partir de la même expression, qui définit l'état « tapis en marche » :

```
bool running = beltActive && !paused && !cubeError;
```

Un seul calcul alimente donc les trois sorties : elles ne peuvent pas se contredire.



vers PAUSE : « A » reçu, bouton Pause du tableau de bord
depuis PAUSE : interrupteur ou Play → retour à l'état d'origine

FIGURE 2. Diagramme des états et des transitions.

2.3 Lire un capteur : seuil, front et anti-rebond

2.3.1 Du signal analogique à l'information « cube présent »

Les capteurs laser sont lus par le convertisseur analogique-numérique (ADC) de l'ESP32, qui renvoie une valeur x_{ADC} de 0 à 4095 (12 bits). Le faisceau reçu donne une valeur haute, d'environ 4095 ; un cube qui le coupe la fait chuter à environ 76. Le programme choisit un seuil $S = 2000$ et décide :

$$\text{cube présent} \iff x_{\text{ADC}} < S \qquad \text{(capteur « actif bas »)}. \tag{2}$$

Le seuil se place entre les deux niveaux observés, dont le milieu vaut $(4095 + 76)/2 \approx 2086$. Un indicateur `activeLow` permet d'inverser la logique si le montage est câblé dans l'autre sens.

TABLE 4. États d'un capteur laser et interprétation logicielle.

Situation	x_{ADC}	Comparaison	Cube présent
Faisceau reçu (repos)	≈ 4095	$x_{\text{ADC}} > S$	non
Faisceau coupé par un cube	≈ 76	$x_{\text{ADC}} < S$	oui

2.3.2 Détection de niveau ou de front

Tant que le cube reste devant le capteur, le niveau « présent » reste vrai pendant de nombreux passages de boucle. Si le programme réagissait au niveau, il créerait un nouveau cube à chaque passage. Il faut réagir au *front montant* : l'instant précis où l'état passe de « absent » à « présent ». On mémorise donc l'état précédent et on ne signale que la transition.

2.3.3 Anti-rebond

Quand le faisceau est coupé, la valeur peut osciller autour du seuil pendant un court instant (vibration, bord du cube, bruit électrique). Chaque oscillation franchit le seuil et produirait un faux front. L'anti-rebond consiste à ignorer tout nouveau front pendant une durée Δt_{deb} après le dernier front accepté. Un front est donc validé si, et seulement si :

$$\underbrace{x_{\text{ADC}}(k) < S}_{\text{présent}} \;\wedge\; \underbrace{\neg\,\text{présent}(k-1)}_{\text{absent avant}} \;\wedge\; \underbrace{t_{\text{now}} - t_{\text{dernier}} > \Delta t_{\text{deb}}}_{\text{hors fenêtre}}. \tag{3}$$

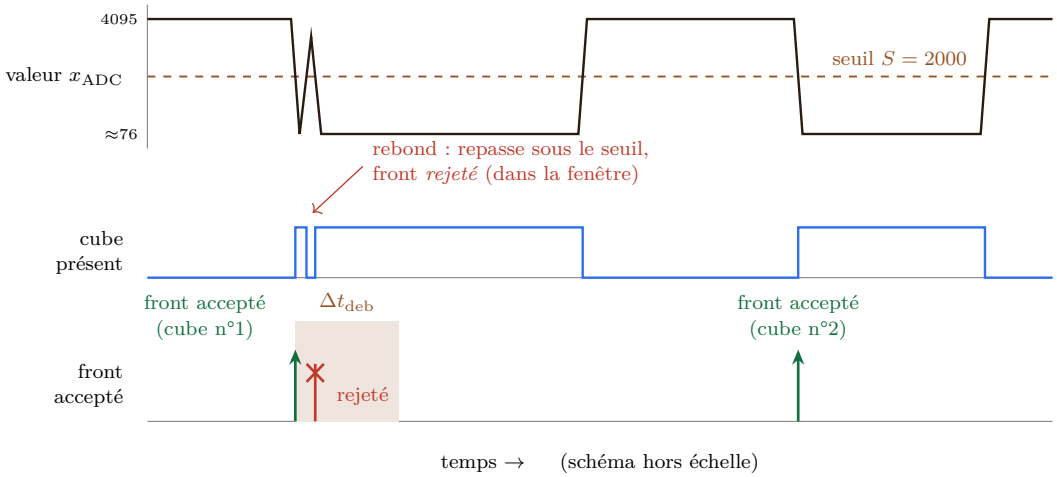


FIGURE 3. Détection de front avec anti-rebond : un rebond à l'arrivée du cube est rejeté.

Dans ce projet. Deux capteurs utilisent ce mécanisme, avec des durées différentes : $\Delta t_{\text{deb}} = 600$ ms au début du tapis et 300 ms en fin de tapis (voir le dimensionnement au section 3.2). Les capteurs sont branchés sur les broches 32 et 33, qui appartiennent au convertisseur ADC1 : sur l'ESP32, l'ADC2 est inutilisable quand le Wi-Fi est actif. Au démarrage, l'état « précédent » est initialisé avec l'état réel des capteurs, ce qui évite de détecter un faux cube si un objet se trouve déjà devant le faisceau.

2.4 La file FIFO circulaire

Plusieurs cubes se trouvent sur le tapis en même temps, chacun avec sa propre couleur et son propre état. Comme le tapis est linéaire, le premier cube entré est aussi le premier à arriver en bout de course : c'est le principe d'une file FIFO (premier entré, premier sorti).

2.4.1 Tableau statique et indices

Plutôt qu'une allocation dynamique, qui fragmente la mémoire au fil du temps, on utilise un tableau de taille fixe $N_{\max}$ et trois entiers : `queueHead` (indice du plus ancien élément), `queueTail` (indice de la prochaine case libre) et `cubesOnBelt` (nombre d'éléments). Quand un indice atteint la fin du tableau, il revient à 0 : c'est le « modulo » qui rend la file circulaire.

$$\text{ajout :} \quad \text{tab[tail]} \leftarrow x; \quad \text{tail} \leftarrow (\text{tail} + 1) \bmod N_{\max}; \quad \text{n} \leftarrow \text{n} + 1 \tag{4}$$

$$\text{retrait :} \quad \text{head} \leftarrow (\text{head} + 1) \bmod N_{\max}; \quad \text{n} \leftarrow \text{n} - 1 \tag{5}$$

$$\text{élément n°}i\text{ :} \quad \text{indice} = (\text{head} + i) \bmod N_{\max} \tag{6}$$

Dans ce projet. Chaque case contient une structure `Cube` de 12 octets, soit 120 octets pour toute la file. La taille de 10 cubes est un compromis : au-delà, il faudrait un tapis bien plus long que celui du projet (voir section 3.3). Retirer un cube au milieu de la file, ce qui arrive après une erreur de cube inconnu, se fait en décalant d'une case les suivants.

2.5 Générer des signaux avec le PWM matériel

Le PWM (modulation de largeur d'impulsion) est un signal carré caractérisé par sa fréquence et son rapport cyclique. L'ESP32 possède un périphérique dédié, le LEDC, qui génère ces

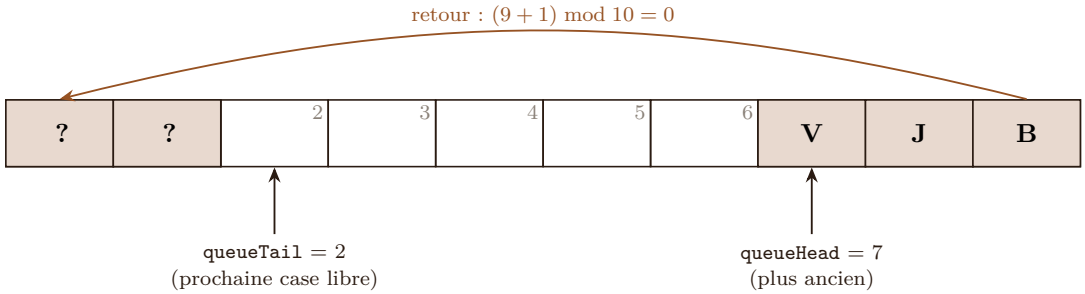


FIGURE 4. File circulaire de $N_{\max} = 10$ cases contenant cinq cubes : `cubesOnBelt` $= 5$. Les cases 7, 8 et 9 puis 0 et 1 sont occupées ; `?` désigne une couleur non encore reçue.

signaux sans intervention du processeur : on lui donne une fréquence, une résolution en bits et une valeur de rapport cyclique, puis il travaille seul. Un rapport cyclique codé sur R bits va de 0 à $2^R - 1$.

Deux usages différents coexistent dans le projet :

— **Train d'impulsions pour le moteur** : la fréquence est la grandeur utile (chaque impulsion = un pas). On fixe la fréquence avec `ledcWriteTone()`, le rapport cyclique restant par défaut.

— **Position du servomoteur** : la fréquence est fixe (50 Hz) et c'est la *largeur* de l'impulsion qui fixe l'angle. Avec $R = 14$ bits :

$$\text{duty} = \frac{t_{\text{imp}}}{T_{\text{PWM}}}\,(2^R - 1) = \frac{t_{\text{imp}}}{20\,000\ \mu\text{s}} \times 16\,383. \tag{7}$$

```
ledcAttachChannel(PIN_PUL,   1000, 8, 0);   // motor pulses, channel 0
ledcAttachChannel(PIN_SERVO,   50, 14, 2);  // servo, 14 bits, channel 2

ledcWriteTone(PIN_PUL, (uint32_t)stepFrequencyHz());   // motor: set frequency
ledcWrite(PIN_SERVO, duty);                            // servo: set pulse width
```

Dans ce projet. Le programme de test du moteur (`02_stepper_tb6600`) produit chaque impulsion à la main avec `micros()` : le processeur est occupé en permanence par le moteur. Dans le programme principal, le LEDC remplace cette méthode : le processeur écrit la fréquence une fois, puis reste libre pour les capteurs. Les canaux sont choisis à la main (0 pour le moteur, 2 pour le servo), car les canaux 0 et 1 partagent le même timer, et le moteur et le servomoteur ont des fréquences différentes.

2.6 La liaison série UART

L'UART est une liaison série asynchrone : l'émetteur et le récepteur n'ont pas d'horloge commune mais s'accordent sur une même vitesse (en bauds). Une trame « 8N1 » envoie 8 bits de données, sans bit de parité, avec 1 bit d'arrêt ; en ajoutant le bit de départ, un octet occupe 10 bits. À 9600 bauds :

$$\text{débit} = \frac{9600}{10} = 960 \text{ octets/s}, \qquad \text{durée d'un octet} = \frac{10}{9600} \approx 1{,}04 \text{ ms}. \tag{8}$$

La caméra envoie un unique caractère par message, ce qui évite tout besoin de découpage de trame ou de somme de contrôle (tableau 5). Côté ESP32, la lecture ne bloque jamais : on vide ce qui est disponible, puis on continue.

TABLE 5. Protocole de la liaison caméra → ESP32.

Octet	Signification	Effet
`J`	Jaune	Couleur attribuée au plus ancien cube sans couleur
`V`	Vert	idem
`B`	Bleu	idem
`X`	Inconnu	idem, plus trois clignotements d'alerte ; le cube arrêtera le tapis en fin de course
`A`	Arrêt d'urgence	Passage immédiat en PAUSE
autres (retour chariot, etc.)	Sans objet	Ignorés

Dans ce projet. Le code accepte les minuscules (conversion par `toupper`). Le même point d'entrée, la fonction `receiveByte()`, sert pour l'UART et pour le clavier du moniteur série : on peut donc tester tout le tri sans caméra en tapant J, V, B, X ou A. Enfin, un octet perdu n'est pas dangereux : le cube reste à la couleur `?`, traitée comme inconnue, ce qui provoque un arrêt sûr en bout de tapis plutôt qu'un tri dans le mauvais bac.

2.7 Le bus I2C et l'écran LCD

L'I2C relie un maître à plusieurs périphériques avec deux fils seulement : SDA (données) et SCL (horloge). Chaque périphérique répond à une adresse ; celle de l'écran est `0x27`. L'écran 16×2 affiche deux lignes de 16 caractères ; les tampons du programme comptent donc 17 octets (16 caractères et le caractère de fin de chaîne), et `snprintf()` tronque proprement tout message trop long.

Dans ce projet. Trois protections rendent l'écran fiable dans un environnement où les moteurs génèrent du bruit électrique : le bus est réglé à 100 kHz, vitesse plus tolérante au bruit ; un délai maximal de 100 ms (`Wire.setTimeOut(100)`) empêche un blocage du bus de figer le programme ; et le dernier message est conservé en mémoire, puis réécrit en entier toutes les 5 s après réinitialisation de l'écran, ce qui efface d'éventuels caractères parasites.

2.8 Multitâche : deux cœurs, une tâche, une file de messages

2.8.1 Le problème

La bibliothèque Blynk doit être appelée régulièrement (`Blynk.run()`), et cet appel peut durer longtemps quand le Wi-Fi est lent ou se reconnecte. Si on l'appelait dans la boucle principale, un réseau capricieux immobiliserait le tapis et les capteurs.

2.8.2 La solution

L'ESP32 a deux cœurs et un système temps réel, FreeRTOS, qui permet de lancer des *tâches* indépendantes. La boucle `loop()` s'exécute sur le cœur 1 ; une tâche dédiée à Blynk est épinglée sur le cœur 0. Les deux ne communiquent que par deux mécanismes (figure 5) :

— **une file de messages** (*queue*) pour aller de `loop()` vers le cloud : `loop()` dépose un message, la tâche Blynk le retire et l'envoie. C'est le schéma producteur–consommateur. L'écriture `xQueueSend(..., 0)` a un délai d'attente nul : si la file est pleine, le message

est perdu mais `loop()` n'attend jamais ;

— **des drapeaux** `volatile` pour aller du cloud vers `loop()` : les fonctions `BLYNK_WRITE` (exécutées dans la tâche Blynk) positionnent des variables, que `loop()` lit et remet à zéro. Le mot-clé `volatile` interdit au compilateur de garder la variable dans un registre : sans lui, `loop()` pourrait ne jamais voir la modification faite par l'autre cœur.

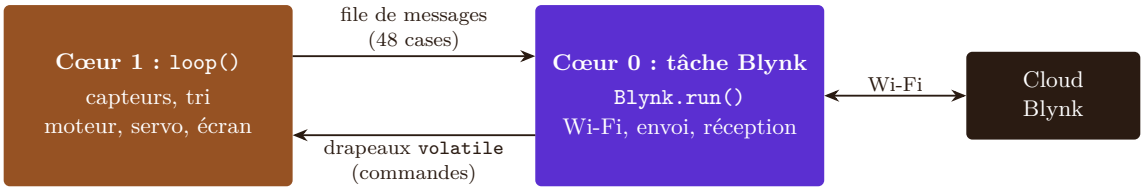


FIGURE 5. Répartition du travail entre les deux cœurs et moyens de communication.

2.8.3 Ordre d'écriture

Le code d'un message du tableau de bord (un texte) est copié dans un tampon, puis le drapeau « code reçu » est positionné *en dernier*. Ainsi `loop()`, qui teste d'abord le drapeau, ne lit jamais un tampon à moitié écrit. Cette règle (« les données d'abord, le drapeau ensuite ») suffit ici parce qu'il n'y a qu'un écrivain et qu'un lecteur par variable ; avec plusieurs écrivains, il faudrait un verrou (*mutex*).

2.9 Mémoire non volatile : `Preferences`

La mémoire vive est effacée à chaque coupure d'alimentation. Pour que les compteurs de tri survivent à un redémarrage, ils sont enregistrés dans la mémoire flash, par la bibliothèque `Preferences` (système NVS). Elle fonctionne par paires clé–valeur, regroupées dans un espace de noms.

```
prefs.begin("tri", false);          // namespace "tri", read/write
cntJ = prefs.getUInt("J", 0);       // read, 0 if the key does not exist
prefs.putUInt("J", cntJ);           // write
```

La flash supporte un nombre limité de cycles d'écriture (de l'ordre de 100 000 cycles d'effacement par secteur pour les mémoires de ce type) ; le système NVS répartit les écritures sur plusieurs secteurs pour retarder l'usure. On évite néanmoins d'écrire inutilement : dans ce projet, l'écriture n'a lieu qu'à chaque cube trié ou à chaque remise à zéro, jamais à chaque tour de boucle.

2.10 Supervision à distance avec Blynk

Blynk est une plateforme qui fournit un tableau de bord web et mobile sans serveur à maintenir. Elle repose sur quatre notions :

— un **modèle** (*template*) qui décrit le produit, identifié par `BLYNK_TEMPLATE_ID` et `BLYNK_TEMPLATE_NAME` ;

— un **jeton d'authentification** (`BLYNK_AUTH_TOKEN`) qui associe l'appareil au modèle ; ces trois définitions doivent apparaître avant les `##include` de Blynk ;

— des **broches virtuelles** V0, V1, … : des canaux de données, chacun relié à un composant (*widget*) du tableau de bord ;

— deux sens d'échange (tableau 6).

Table 6. Les deux sens d'échange avec Blynk.

Sens	Mécanisme	Exemple dans le projet
Montant (ESP32 → cloud)	`Blynk.virtualWrite(pin, valeur)`, appelé par la tâche Blynk après passage par la file	Compteurs sur V1, V5 à V8
Descendant (cloud → ESP32)	`BLYNK_WRITE(Vn)` : fonction rappelée à chaque changement du composant	Bouton Play/Pause sur V10

Le service limite le nombre de messages (environ 10 par seconde). Deux règles en découlent : espacer les envois (ici de 110 ms au minimum) et n'envoyer l'état des LED et du bouton que lorsqu'il *change*.

2.11 Heure réseau et journal d'événements

L'ESP32 n'a pas d'horloge de calendrier : il ne connaît que le temps écoulé depuis son démarrage. Pour horodater les événements du journal du tableau de bord, il interroge un serveur de temps (NTP) par `configTime()`, avec le décalage du Bénin, UTC+1 sans heure d'été (3 600 s). La lecture `getLocalTime(&t, 0)` a un délai d'attente nul : si l'heure n'est pas encore connue, le programme n'attend pas et affiche à la place le temps écoulé depuis le démarrage, sous la forme « +HH :MM :SS ». Sur le port série, chaque ligne commence par le temps en millisecondes grâce à la macro `LOG()`.

PARTIE 3 : MODULES DU PROGRAMME

Chaque module est présenté selon le même plan : rôle, outils utilisés (fonctions et structures du code), schéma, fonctionnement et dimensionnement. Les noms des fonctions et des variables sont ceux du fichier source, en anglais, pour que le lecteur puisse les retrouver directement dans le code.

3.1 Module 1 : Configuration et paramètres

Rôle

Ce module regroupe en tête de fichier tout ce qui peut se régler : brochage, vitesse, géométrie, seuils, durées, identifiants réseau. La logique n'utilise jamais de valeur numérique « en dur » : elle utilise un nom de constante. Régler le système ne demande donc de modifier que cette zone, sans toucher aux algorithmes.

Outils utilisés

Des constantes déclarées `const`, regroupées par thème (moteur, géométrie, servomoteur, capteurs, sécurités, signalisation, débogage, réseau). Les trois identifiants Blynk sont définis avant les `##include`, comme l'exige la bibliothèque.

Fonctionnement

Les constantes sont évaluées à la compilation et ne coûtent pas de mémoire vive. Les seules grandeurs modifiables à l'exécution sont la vitesse (`rpm`, changée par `setRpm()`) et l'état du système. Des indicateurs booléens activent ou non des fonctions de débogage : `KEYBOARD_SIMULATION`, `DEBUG_RAW_UART`, `WATCHDOG_ENABLED`.

Dimensionnement

Tableau 7 liste les paramètres principaux et la raison de leur valeur ; le brochage figure dans la table 15.

Table 7. Paramètres de réglage du programme.

Constante	Valeur	Rôle
`rpm`	24 tr/min	Vitesse de rotation du rouleau
`STEPS_PER_REV_MOTOR`	200	Pas par tour du moteur
`MICROSTEP`	1	Micro-pas réglé sur le driver
`ROLLER_PERIMETER_CM`	10 cm	Périmètre du rouleau (à mesurer)
`END_TO_BIN_DISTANCE_CM`	5 cm	Distance fin de tapis–bac, aussi espacement minimal des cubes
`ANGLE_J / ANGLE_B / ANGLE_V`	0 / 90 / 180°	Angle du servo pour jaune, bleu, vert
`ANGLE_HOME`	90°	Position au démarrage et après un arrêt
`DETECTION_THRESHOLD`	2000	Seuil S de détection (sur 4095)
`DEBOUNCE_START_MS`	600 ms	Anti-rebond du capteur de début
`DEBOUNCE_END_MS`	300 ms	Anti-rebond du capteur de fin
`START_DELAY_MS`	300 ms	Délai entre la détection et le démarrage du tapis
`INACTIVITY_TIMEOUT_MS`	120 000 ms	Délai du chien de garde
`TOGGLE_DEBOUNCE_MS`	50 ms	Anti-rebond de l'interrupteur de reprise
`EMERGENCY_HALF_PERIOD_MS`	250 ms	Demi-période du clignotement pendant l'arrêt d'urgence
`ALERT_HALF_PERIOD_MS`	200 ms	Demi-période du clignotement d'alerte
`ALERT_BLINK_COUNT`	3	Clignotements à la réception d'un `X`
`CLEAR_SAMPLES_REQUIRED`	5	Lectures « libre » consécutives pour lever une erreur
`LCD_REFRESH_MS`	5000 ms	Réécriture complète de l'écran
`HEARTBEAT_MS`	5000 ms	Période de la ligne d'état sur le port série
`BLYNK_INTERVAL_MS`	110 ms	Intervalle minimal entre deux envois au cloud
`MAX_CUBES_ON_BELT`	10	Capacité de la file
`RESET_CODE`	« EPAC »	Code de remise à zéro des compteurs

3.2 Module 2 : Acquisition des capteurs laser

Rôle

Transformer la valeur analogique de chaque capteur en un événement fiable : « un cube vient d'arriver devant ce faisceau ». Ce module ne décide rien : il fournit les événements aux autres modules.

Outils utilisés

La structure `Sensor` (broche, durée d'anti-rebond, dernier instant accepté, état précédent, dernière valeur, sens), les deux instances `sensorStart` et `sensorEnd`, et trois fonctions : `cubePresent()`, `risingEdge()` et `initSensors()`. L'étage physique (laser, photorésistance) est décrit dans l'étude électrique, sous-systèmes 1 et 2.

Schéma

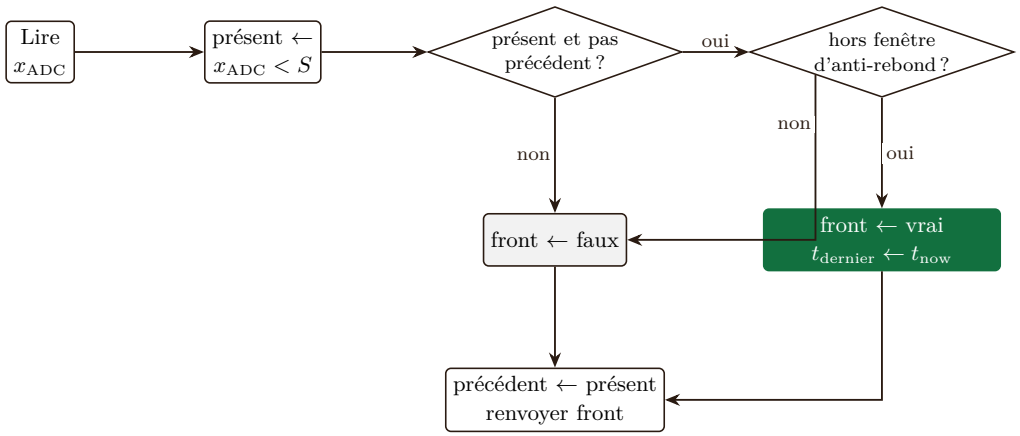


FIGURE 6. Algorithme de `risingEdge()` : détection d'un front avec anti-rebond.

Fonctionnement

À chaque passage de boucle, `risingEdge()` relit l'entrée, détermine si un cube est présent, puis applique les trois conditions de l'équation du front : présent, absent au tour précédent, hors fenêtre d'anti-rebond. L'état « précédent » est mis à jour à *chaque* appel, qu'il y ait front ou non ; sans cela, un cube resté devant le capteur serait vu comme un nouveau front à la première occasion.

```
bool risingEdge(Sensor &s) {
  s.value = analogRead(s.pin);
  bool present = cubePresent(s);
  bool edge = present && !s.previous && elapsedSince(s.lastMs) > s.debounceMs;
  if (edge) s.lastMs = millis();
  s.previous = present;
  return edge;
}
```

Le programme appelle aussi `risingEdge()` pendant l'arrêt d'urgence, en ignorant son résultat : les états « précédents » restent ainsi à jour, et un cube qui reste devant un capteur pendant la pause ne produit pas de faux front à la reprise.

Dimensionnement

Seuil. $S = 2000$ se situe entre le niveau haut (≈ 4095) et le niveau bas (≈ 76), près de leur milieu (≈ 2086), ce qui laisse une marge de plus de 1900 unités de chaque côté pour absorber le bruit.

Anti-rebond. Il doit rester inférieur au temps minimal qui sépare deux cubes devant un même capteur, sinon un vrai cube serait rejeté. Avec la vitesse $v = 4$ cm/s (voir le module 5) et l'espacement minimal $D = 5$ cm :

$$T_{\min} = \frac{D}{v} = \frac{5}{4} = 1{,}25 \text{ s} = 1250 \text{ ms} \quad \Rightarrow \quad \Delta t_{\text{deb}} = 600 \text{ ms} < 1250 \text{ ms} \ \text{ et } \ 300 \text{ ms} < 1250 \text{ ms}. \tag{9}$$

La contrainte est respectée avec une marge d'un facteur supérieur à deux. Elle se resserre si la vitesse augmente : à 48 tr/min, $T_{\min} = 625$ ms, et les 600 ms du capteur de début deviennent presque insuffisantes.

3.3 Module 3 : File des cubes et attribution des couleurs

Rôle

Suivre chaque cube présent sur le tapis, depuis sa détection au début jusqu'à son tri, en lui associant sa couleur et son état d'avancement. C'est la mémoire du système : tous les autres modules la consultent.

Outils utilisés

La structure `Cube` et la file circulaire `cubeQueue` (voir la partie 2), avec les fonctions `addNewCube()`, `receiveColor()`, `handleEndOfBelt()`, `advanceToNextCube()`, `removeCubeAt()` et `resetQueue()`.

TABLE 8. Champs de la structure `Cube`.

Champ	Type	Signification
`color`	`char`	`J`, `V`, `B`, `X` ou `?` (pas encore reçue)
`reachedEnd`	`bool`	Le capteur de fin a vu ce cube
`binAligned`	`bool`	Le bac a été orienté pour ce cube
`entryTime`	`unsigned long`	Instant de détection au début (sert à la durée totale de transit)
`dropTime`	`unsigned long`	Instant d'arrivée en fin de tapis, point de départ du délai de chute

Schéma

Fonctionnement

Création. À chaque front du capteur de début, `addNewCube()` ajoute en queue un cube de couleur `?`, dont `entryTime` vaut l'instant courant. Si la file est pleine, le cube est ignoré et signalé dans le journal ; si le tapis est arrêté, le démarrage différé est programmé.

Attribution de la couleur. Quand un octet de couleur arrive, `receiveColor()` parcourt la file depuis la tête et donne la couleur au *plus ancien cube encore à* `?`. C'est le même raisonnement FIFO que pour l'ordre d'arrivée : la caméra identifie les cubes dans l'ordre où ils sont entrés. Si aucun cube n'attend de couleur, l'octet est ignoré.

Arrivée en fin de tapis. Quand le laser de fin détecte un cube, `handleEndOfBelt()` cherche

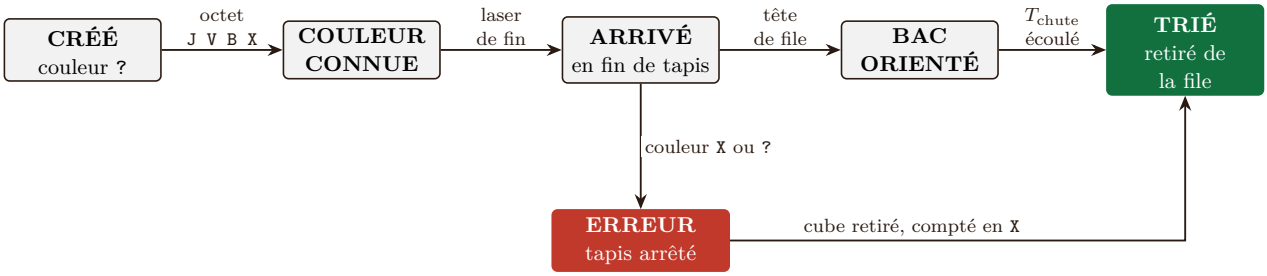


FIGURE 7. Cycle de vie d'un cube dans la file. L'orientation du bac a lieu à l'arrivée si le cube est en tête de file, sinon dès que le cube précédent est trié.

le premier cube de la file dont `reachedEnd` est faux : c'est celui qui arrive, puisque les cubes arrivent dans l'ordre. Trois cas se présentent :

1. couleur inconnue (`X` ou `?`) : arrêt immédiat du tapis (module 7) ;
2. cube de tête (position 0) : le bac est orienté tout de suite ;
3. cube en position $i > 0$: l'orientation est *différée*, car le cube précédent est encore en train de tomber et changer le bac le dirigerait dans le mauvais récipient. Elle sera faite par `advanceToNextCube()` dès que le cube de tête sera trié.

```
for (int i = 0; i < cubesOnBelt; i++) {
  int idx = (queueHead + i) % MAX_CUBES_ON_BELT;
  if (!cubeQueue[idx].reachedEnd) {      // first cube not yet seen at the end
    cubeQueue[idx].reachedEnd = true;
    cubeQueue[idx].dropTime = millis();
    /* unknown colour -> error; head of queue -> orient bin; else postpone */
    return;
  }
}
```

Dimensionnement

Mémoire. Une structure `Cube` occupe 12 octets (un caractère, deux booléens, un octet de remplissage et deux entiers de 4 octets) : la file complète de 10 cubes occupe 120 octets.

Capacité. Avec un espacement minimal $D = 5$ cm entre cubes, une file de $N_{\max} = 10$ cubes suffit pour un tapis d'environ $N_{\max} \times D = 50$ cm. Pour un tapis plus long, il faut augmenter `MAX_CUBES_ON_BELT` ; la mémoire nécessaire reste négligeable.

3.4 Module 4 : Réception des ordres de la caméra

Rôle

Lire sur la liaison UART les octets envoyés par la carte XIAO, et les transformer en actions : attribuer une couleur ou déclencher l'arrêt d'urgence.

Outils utilisés

Le port série `Serial1` (RX sur la broche 16, 9600 bauds, 8N1) et les fonctions `handleCameraUart()`, `readCameraByte()`, `receiveByte()` et `receiveColor()`. Le port `Serial` (USB) sert au clavier de simulation.

Schéma

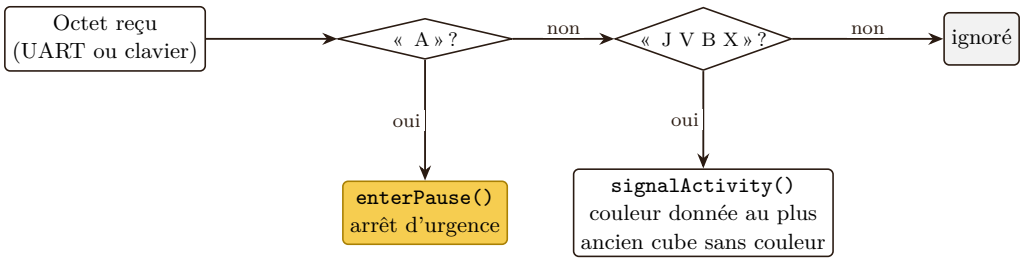


FIGURE 8. Aiguillage d'un octet reçu.

Fonctionnement

`handleCameraUart()` lit tous les octets disponibles (boucle `while`) puis rend la main : elle ne bloque jamais. Chaque octet passe par `receiveByte()`, qui teste d'abord le code d'arrêt `A`, prioritaire, puis transmet à `receiveColor()`. Celle-ci met l'octet en majuscule, élimine tout ce qui n'est pas `J`, `V`, `B` ou `X`, et déclare une activité au chien de garde. Un `X` déclenche en plus trois clignotements d'alerte.

Pendant l'arrêt d'urgence, les octets reçus sont lus puis jetés. C'est volontaire : un octet de couleur arrivé pendant la pause serait attribué à un cube qui n'a peut-être plus la même place à la reprise, ce qui décalerait toutes les associations couleur–cube suivantes.

Dimensionnement

À 9600 bauds, un octet dure environ 1,04 ms. Il arrive donc au plus un octet toutes les millisecondes, et l'envoi d'une couleur par cube, à raison d'un cube toutes les 1,25 s au minimum, est très inférieur à la capacité de la liaison. Le tampon de réception du pilote (256 octets par défaut) ne peut pas déborder tant que la boucle principale tourne plus vite que la caméra n'émet.

3.5 Module 5 : Commande du moteur du tapis

Rôle

Faire avancer le tapis à vitesse constante, seulement quand il y a des cubes à trier, et calculer le temps de chute qui découle de cette vitesse.

Outils utilisés

Le signal d'impulsions PUL produit par le LEDC (broche 5), la broche d'activation ENA (broche 2) et les fonctions `startMotor()`, `stopMotor()`, `setRpm()`, `stepFrequencyHz()` et `computeDropDelayMs()`. Le sens de rotation n'est pas commandé : l'entrée DIR du driver est câblée directement à l'alimentation, le tapis n'avançant que dans un sens.

Fonctionnement

Au démarrage, la broche ENA est mise au niveau qui *active* le driver (`ENA_ACTIVE_LEVEL`, dépendant du câblage) et reste ainsi ; seul le signal PUL commande le mouvement. `startMotor()` règle la fréquence d'impulsions avec `ledcWriteTone()`, et `stopMotor()` la met à zéro.

Le démarrage est *différé* : à la détection du premier cube, on lève le drapeau `beltStartPending` et on mémorise l'instant ; la boucle principale lance le moteur lorsque

`START_DELAY_MS` (300 ms) s'est écoulé. Le délai est donc obtenu sans `delay()`, et la boucle reste à l'écoute pendant ce temps. Le moteur s'arrête normalement quand la file est vide (`advanceToNextCube()`), et en urgence dans les cas du module 7.

Dimensionnement

Trois grandeurs découlent de la vitesse n du rouleau :

$$f_{\text{pas}} = \frac{n \cdot N_{\text{rev}} \cdot \mu}{60}, \tag{10}$$

$$v = \frac{n}{60} P, \tag{11}$$

$$T_{\text{chute}} = \frac{D}{v} \times 1000 = \frac{60\,D}{n\,P} \times 1000 \text{ ms}. \tag{12}$$

Avec les valeurs du projet ($n = 24$ tr/min, $N_{\text{rev}} = 200$, $\mu = 1$, $P = 10$ cm, $D = 5$ cm) :

$$f_{\text{pas}} = \frac{24 \times 200 \times 1}{60} = 80 \text{ Hz}, \qquad v = \frac{24}{60} \times 10 = 4 \text{ cm/s}, \qquad T_{\text{chute}} = \frac{5}{4} \times 1000 = 1250 \text{ ms}. \tag{13}$$

TABLE 9. Effet de la vitesse sur la fréquence de pas, la vitesse du tapis et le délai de chute.

n **(tr/min)**	f_{pas} **(Hz)**	v **(cm/s)**	T_{chute} **(ms)**
12	40	2	2500
24	80	4	1250
48	160	8	625

Le délai de chute est inversement proportionnel à la vitesse : doubler la vitesse divise par deux le temps dont dispose le bac pour s'orienter. Le calcul est donc refait à chaque usage à partir de la valeur courante de `rpm`, plutôt que fixé une fois pour toutes. Si `rpm` est nul ou négatif, la fonction renvoie un délai de repli de 2000 ms pour éviter une division par zéro. Le résultat est tronqué à la milliseconde entière, ce qui est négligeable devant les 1250 ms.

3.6 Module 6 : Orientation du bac par le servomoteur

Rôle

Placer le bac sous la sortie du tapis, dans la position qui correspond à la couleur du cube qui va tomber, et ne jamais le déplacer tant qu'un cube est en train de tomber.

Outils utilisés

Le signal PWM à 50 Hz sur la broche 23 et les fonctions `servoAngle()`, `angleFor()`, `orientBin()`, `menuIndex()` et `colorName()`. Le tableau `BIN_MENU` donne l'ordre des choix de la liste déroulante « Bac » du tableau de bord.

Schéma

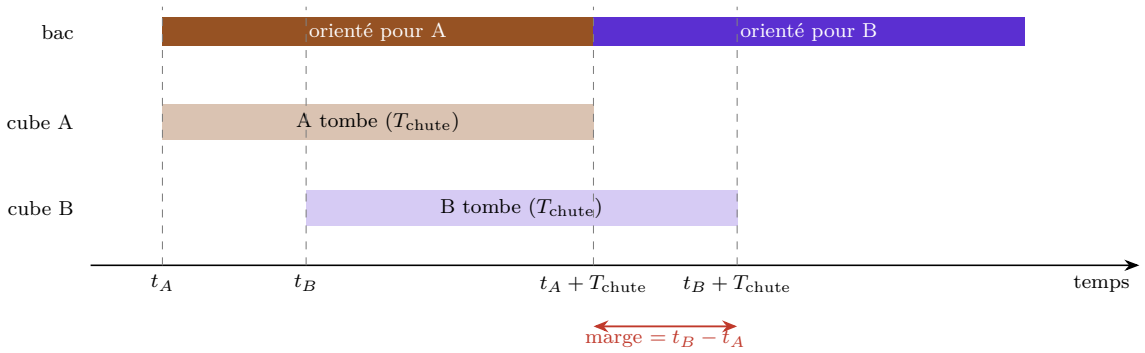


FIGURE 9. Orientation différée du bac pour deux cubes proches : le bac change seulement quand A a fini de tomber.

Fonctionnement

`servoAngle()` limite l'angle entre 0 et 180°, calcule la largeur d'impulsion

$$t_{\text{imp}} = 500 + \frac{\text{angle}}{180} \times 1900 \quad (\mu\text{s}), \tag{14}$$

puis la convertit en rapport cyclique par l'équation (7). `orientBin()` traduit une couleur en angle (`angleFor()`, qui renvoie -1 pour une couleur non triable et n'oriente alors rien), positionne le servo, mémorise la couleur du bac, met à jour la liste déroulante du tableau de bord et affiche la couleur sur l'écran.

La figure 9 montre la règle essentielle : le bac suit l'ordre des cubes. Quand deux cubes arrivent en fin de tapis à t_A puis t_B avec $t_B - t_A < T_{\text{chute}}$, A est encore en train de tomber à l'arrivée de B. Le bac reste donc sur la position de A jusqu'à l'instant $t_A + T_{\text{chute}}$; il est alors orienté pour B, qui tombera à $t_B + T_{\text{chute}}$. La marge dont dispose le servo pour se déplacer est $t_B - t_A$.

Dimensionnement

Largeurs d'impulsion. Le code utilise une plage de 500 à 2400 μs, plus large que la plage nominale de 1 à 2 ms donnée pour le SG90 dans l'étude électrique : elle permet de couvrir toute la course de 180°. Pour les trois positions du projet :

TABLE 10. Positions du bac : angle, largeur d'impulsion et rapport cyclique codé sur 14 bits.

Couleur	Angle	t_{imp} (μs)	Calcul	duty
Jaune (`J`)	0°	500	$500/20000 \times 16383 = 409{,}6$	409
Bleu (`B`)	90°	1450	$1450/20000 \times 16383 = 1187{,}8$	1187
Vert (`V`)	180°	2400	$2400/20000 \times 16383 = 1966{,}0$	1965

La conversion en entier tronque : la dernière ligne vaut 1965 et non 1966, soit un écart inférieur à 1,3 μs.

Résolution. Un pas de rapport cyclique vaut $20\,000/16\,383 \approx 1{,}22$ μs, soit $1{,}22 \times 180/1900 \approx 0{,}12°$: très inférieure à la précision mécanique du servomoteur.

Temps de réponse. La marge disponible pour déplacer le servo est au moins $D/v = 1250$ ms quand les cubes respectent l'espacement minimal. Elle est largement supérieure au temps de parcours typique d'un SG90 (de l'ordre de 0,1 s pour 60°). Le programme n'attend pas la fin du mouvement : il compare simplement l'instant courant à `dropTime` et signale dans le journal la marge restante, ou un avertissement « cubes trop rapprochés ».

3.7 Module 7 : Sécurités

Rôle

Garantir qu'en toute circonstance le système s'arrête sans perdre le suivi des cubes : arrêt d'urgence, reprise contrôlée, cube inconnu, inactivité prolongée.

Outils utilisés

Les drapeaux `paused`, `cubeError`, les instants `pauseStart` et `errorStart`, la broche `PIN_PAUSE` (interrupteur), et les fonctions `enterPause()`, `exitPause()`, `handleToggleSwitch()`, `triggerCubeError()`, `handleCubeError()` et `signalActivity()`.

TABLE 11. Les quatre sécurités du système.

Sécurité	Déclencheur	Action	Sortie de l'état
Arrêt d'urgence	Octet `A` (bouton de la caméra), bouton Pause du tableau de bord, touche `A` du clavier	Moteur coupé, tout est ignoré, LED rouge clignotante	Changement de position de l'interrupteur, ou Play
Cube inconnu	Cube de couleur `X` ou `?` détecté en fin de tapis	Moteur coupé sur place, LED rouge clignotante	Capteur de fin libre (retrait manuel), reprise automatique
Chien de garde	120 s sans aucun signe d'activité tapis en marche	Arrêt, bac au centre, file vidée	Nouveau cube au début du tapis
Anti-rebond	Faux fronts (capteurs, interrupteur)	Ignorés	Sans objet

Fonctionnement

Arrêt d'urgence. `enterPause()` lève `paused`, note l'instant, coupe le moteur et affiche « EMERGENCY STOP ». Dans la boucle principale, ce drapeau est testé avant toute autre action : tant qu'il est levé, seuls l'interrupteur, les commandes du tableau de bord et les LED sont traités ; les octets reçus sont jetés et les capteurs lus à blanc.

Gel des chronomètres. Un cube a un délai de chute en cours quand l'arrêt survient ; ce délai ne doit pas continuer à s'écouler pendant la pause, sinon le cube serait déclaré trié alors qu'il n'a pas bougé. À la reprise, `exitPause()` calcule la durée $T_{\text{pause}} = t_{\text{reprise}} - t_{\text{pause}}$ et la *reporte* sur tous les instants mémorisés :

$$t'_{\text{mémorisé}} = t_{\text{mémorisé}} + T_{\text{pause}} \quad \Rightarrow \quad t_{\text{now}} - t'_{\text{mémorisé}} \text{ inchangé.} \tag{15}$$

Sont décalés : `entryTime` et `dropTime` de chaque cube, l'instant de démarrage différé et l'instant de début d'erreur. Le temps qui s'est écoulé « pour le système » est ainsi identique avant et après la pause (figure 10).

Reprise par l'interrupteur. La reprise n'exige pas une position précise : elle réagit à *tout changement de niveau* de l'interrupteur, après anti-rebond. L'opérateur n'a donc pas besoin de savoir dans quelle position il se trouve. L'anti-rebond mémorise le niveau « brut » et le niveau « stable » : le niveau stable ne change que si le niveau brut diffère et est resté inchangé pendant au moins 50 ms. Au démarrage, le niveau initial est relu, pour ne pas confondre la position de départ avec une commande.

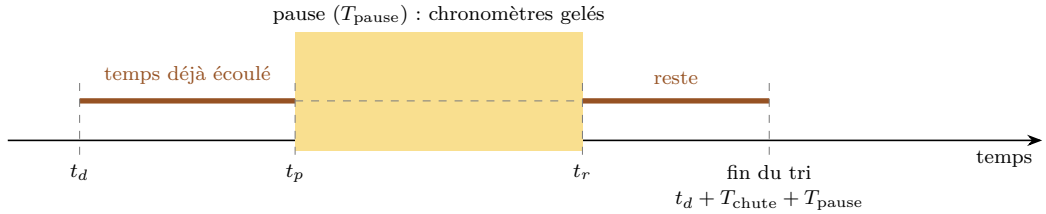


FIGURE 10. Gel des chronomètres : le délai de chute ne compte pas le temps passé en pause.

Cube inconnu. Quand un cube de couleur `X` ou `?` atteint le laser de fin, `triggerCubeError()` coupe le moteur immédiatement : le cube se trouve alors encore devant le capteur, ce qui permet à l'opérateur de le retirer. La fonction `handleCubeError()` attend que le capteur de fin soit libre pendant `CLEAR_SAMPLES_REQUIRED` (5) lectures consécutives. Elle compte alors le cube dans le compteur des inconnus, le retire de la file, reporte les chronomètres des autres cubes de la durée de l'erreur, prépare le cube suivant et relance le moteur s'il reste des cubes.

Chien de garde. À chaque signe de vie (front d'un capteur, couleur reçue, cube trié, reprise, démarrage du tapis, fin d'erreur), `signalActivity()` mémorise l'instant et la source. Si le tapis est actif et que 120 s s'écoulent sans aucun signe de vie, le système suppose une situation anormale (cube coincé hors des capteurs, caméra muette) : il arrête le moteur, remet le bac en position centrale, vide la file, affiche « EMERGENCY » et journalise la cause.

Dimensionnement

Chien de garde. Sa durée doit être supérieure à l'intervalle le plus long entre deux signes de vie en fonctionnement normal. Dans ce projet, le plus long intervalle est le transit d'un cube seul, du capteur de début au capteur de fin ; à la vitesse de 4 cm/s, une longueur de tapis L donne $L/4$ s. À 4 cm/s, 120 s correspondent à 480 cm de tapis parcourus : la marge est très grande. Elle ne doit être réduite qu'après mesure du transit réel.

Lecture multiple. Cinq lectures consécutives évitent qu'une seule lecture « libre » parasite, par exemple pendant que le cube est retiré de biais, relance le tapis trop tôt.

Interrupteur. 50 ms d'anti-rebond sont très supérieures aux rebonds mécaniques usuels (quelques millisecondes) et restent imperceptibles pour un opérateur.

3.8 Module 8 : Signalisation par LED et écran LCD

Rôle

Montrer à tout moment l'état du système à l'opérateur, sans qu'il ait besoin du tableau de bord.

Outils utilisés

Trois LED (verte, jaune, rouge) sur les broches 27, 26 et 14, l'écran LCD 16×2 en I2C, et les fonctions `updateLeds()`, `updateAlert()`, `triggerAlertX()`, `showLcd()`, `writeLcd()` et `refreshLcd()`.

TABLE 12. Commande des LED selon l'état du système.

État	**Verte**	**Jaune**	**Rouge**	**Blynk V2/V3/V4**
MARCHE	fixe	éteinte	éteinte	255 / 0 / 0
ARRÊT (en attente)	éteinte	fixe	éteinte	0 / 255 / 0
PAUSE	éteinte	fixe	clignote (période 500 ms)	0 / 255 / 255
ERREUR	éteinte	fixe	clignote (période 400 ms)	0 / 255 / 255
Réception d'un `X` (en marche)	fixe	éteinte	3 clignotements (1,2 s)	255 / 0 / 0

Fonctionnement

Les LED sont recalculées à chaque tour de boucle à partir de l'état (tableau 12). La LED d'alerte est un paramètre (`PIN_LED_ALERT`) : elle peut être la rouge ou la jaune.

Le clignotement est une petite machine à états non bloquante : un compteur de demi-cycles restants et un instant de dernier changement. À chaque tour, si une demi-période s'est écoulée, l'état de la LED s'inverse et le compteur diminue. Pour trois clignotements, le compteur est initialisé à $2 \times 3 = 6$ demi-cycles de 200 ms : la séquence dure $6 \times 200 = 1200$ ms. Pendant une erreur, le compteur ne diminue pas, le clignotement continue tant que l'erreur dure.

TABLE 13. Messages affichés sur l'écran LCD (16 caractères par ligne).

Événement	**Ligne 1**	**Ligne 2**
Démarrage	`System ready`	`Waiting...`
Nouveau cube	`New cube`	`On belt: n`
Bac orienté	`BIN:`	couleur
Bac choisi depuis le tableau de bord	`BIN (manual):`	couleur
Cube trié	`ITEM SORTED`	couleur
Arrêt d'urgence	`EMERGENCY STOP`	`Flip switch`
Reprise	`RESUMED`	(vide)
Cube inconnu	`CUBE ERROR`	`Remove the cube`
Cube retiré	`Cube removed`	`Resuming`
Chien de garde	`EMERGENCY`	`Idle 120s`

Dimensionnement

Longueur des messages. Tous les textes de tableau 13 tiennent en 16 caractères au plus (le plus long, `Remove the cube`, en compte 15) ; la copie par `snprintf()` dans des tampons de 17 octets protège contre tout dépassement.

Fréquences de clignotement. Pendant l'arrêt d'urgence, la demi-période de 250 ms donne une période de 500 ms (2 Hz) ; pour l'alerte, la demi-période de 200 ms donne 400 ms (2,5 Hz). Les deux rythmes sont différents pour que l'opérateur distingue visuellement une urgence d'une alerte.

Courant des LED. Chaque LED est alimentée par une broche de l'ESP32 à travers une

résistance série calculée dans l'étude électrique (sous-système 8).

3.9 Module 9 : Supervision par le cloud Blynk

Rôle

Envoyer au tableau de bord l'état du système (compteurs, LED, dernier tri, journal, graphique) et recevoir ses commandes (Play/Pause, choix du bac, remise à zéro), sans jamais ralentir le tri.

Outils utilisés

La tâche `blynkTask()` sur le cœur 0, la file `blynkQueue` de structures `Msg`, les fonctions d'envoi `pushInt()`, `pushTxt()` et `termLog()`, les fonctions de réception `BLYNK_WRITE(V10, V12, V13, V15, V16)` et `BLYNK_CONNECTED()`, et, côté `loop()`, `handleBlynkCommands()`, `sendFullState()` et `syncBlynkState()`.

Fonctionnement

Tâche Blynk. Elle boucle indéfiniment. Si le Wi-Fi est coupé, elle marque le système hors ligne et attend 500 ms. Sinon, elle appelle `Blynk.run()`, puis, au plus toutes les 110 ms, retire un message de la file et l'envoie (`virtualWrite`). Si Blynk est hors ligne, la file est vidée : les messages périmés sont inutiles, l'état complet sera renvoyé à la reconnexion. Chaque tour se termine par une pause de 10 ms, qui laisse le temps à la pile Wi-Fi.

Envoi. `pushInt()` et `pushTxt()` construisent un message et le déposent dans la file sans attente (délai nul). `termLog()` formate une ligne horodatée « [HH :MM :SS] message » pour le journal (V12). Le tri ne subit donc que le coût d'une copie en mémoire.

Envoi sur changement. `syncBlynkState()` garde en mémoire les dernières valeurs des LED virtuelles et du bouton Play/Pause, et n'envoie un message que lorsqu'une valeur change. Cela économise le débit autorisé par le cloud. Un indicateur `forceState` réinitialise cette mémoire pour forcer un renvoi.

Réception. Les fonctions `BLYNK_WRITE` (exécutées dans la tâche Blynk) ne font que positionner un drapeau ou copier une valeur ; le traitement réel est fait par `handleBlynkCommands()`, dans `loop()`, au début de chaque tour. Ce découpage garantit que les actions sur le moteur et le servo ne se produisent que dans un seul fil d'exécution. Les règles de chaque commande figurent au tableau 16.

Resynchronisation. À chaque (re)connexion, `BLYNK_CONNECTED()` lève un drapeau ; `loop()` appelle alors `sendFullState()`, qui renvoie le dernier tri, les compteurs, l'état du bac, remet à zéro le bouton de réinitialisation et la zone de code, puis force le renvoi des LED. Le tableau de bord reflète ainsi l'état réel du système après toute coupure.

Dimensionnement

Débit. L'intervalle de 110 ms limite l'envoi à

$$f_{\text{msg}} = \frac{1}{0{,}110} \approx 9{,}1 \text{ messages/s}, \tag{16}$$

sous la limite d'environ 10 messages par seconde du service.

File de messages. Un message (`Msg`) occupe 108 octets : broche, type, remplissage, entier de 4 octets et texte de 100 caractères. La file de 48 messages occupe donc $48 \times 108 = 5184$ octets, soit environ 5 Ko. À la reconnexion, `sendFullState()` dépose 10 messages (dernier tri, cinq compteurs, bouton de réinitialisation, zone de code, liste du bac, ligne de journal) et

`syncBlynkState()` en ajoute 4 : 14 messages au total, évacués en $14 \times 0{,}110 \approx 1{,}5$ s. La file de 48 cases absorbe cette salve avec une grande marge.

Taille des textes. La ligne de journal est construite dans un tampon de 80 caractères, précédé d'un horodatage de 10 caractères et d'un retour à la ligne ; elle tient dans les 100 caractères d'un message.

Pile de la tâche. 8192 octets sont réservés à la tâche Blynk, valeur usuelle pour la pile réseau.

3.10 Module 10 : Comptage, mémoire permanente et remise à zéro

Rôle

Compter les cubes triés par couleur, conserver ces comptes après une coupure de courant, et permettre de les remettre à zéro de façon sûre.

Outils utilisés

Les compteurs `cntJ`, `cntV`, `cntB`, `cntX`, la bibliothèque `Preferences` (espace de noms `tri`), et les fonctions `recordSort()`, `saveCounters()`, `pushCounters()`, `totalSorted()` et `resetCounters()`.

Fonctionnement

Comptage. `recordSort()` est appelée une fois par cube : lors du tri normal (étape 6 de la boucle) et lors de la sortie d'une erreur de cube inconnu, qui incrémente le compteur `X`. Elle incrémente le compteur de la couleur, enregistre les quatre compteurs en mémoire permanente, mémorise le nom du dernier tri, puis envoie au tableau de bord le dernier tri, les compteurs, le total (graphique V11) et une ligne de journal. Au démarrage, les compteurs sont relus depuis la mémoire : un redémarrage ne perd donc rien.

Remise à zéro par double validation. Effacer le comptage de la production est une action sensible. Elle demande deux gestes distincts sur le tableau de bord (figure 11) : armer l'interrupteur « Reset DB » (V13), puis saisir le code dans la zone « Confirmation Code » (V16). Une saisie du code sans interrupteur armé est ignorée ; un mauvais code annule l'armement. Dans tous les cas, l'interrupteur est remis sur OFF et la zone de saisie est vidée. La comparaison ignore la casse (`strcasecmp`) et les espaces autour du texte.

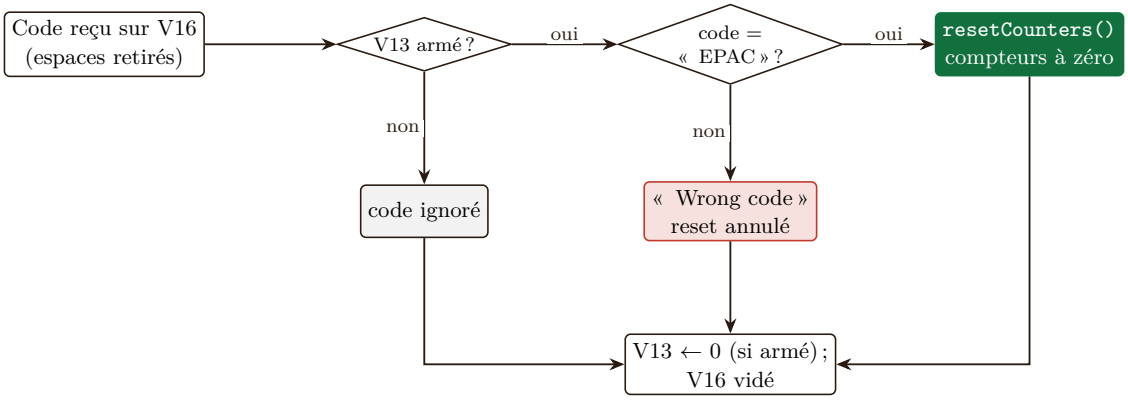


FIGURE 11. Logique de remise à zéro des compteurs.

Dimensionnement

Capacité des compteurs. Ce sont des entiers non signés de 32 bits, jusqu'à $2^{32} - 1 = 4\,294\,967\,295$: le dépassement est hors de portée.

Écritures en flash. Quatre valeurs sont écrites à chaque cube trié. Au rythme maximal d'un cube toutes les 1,25 s (0,8 cube par seconde), cela représente au plus 3,2 écritures de valeur par seconde en régime saturé ; l'usure de la flash devrait être vérifiée si le système fonctionne longtemps en continu (voir la partie 4, limites et perspectives).

PARTIE 4 : INTÉGRATION

4.1 Schéma bloc logiciel

La figure 12 montre comment les dix modules s'enchaînent. Les entrées (capteurs, caméra, interrupteur, tableau de bord) alimentent l'état central du système (la file des cubes, les drapeaux et les chronomètres) ; les modules de sortie en tirent leurs commandes. Les sécurités ont la main sur le moteur à tout moment.

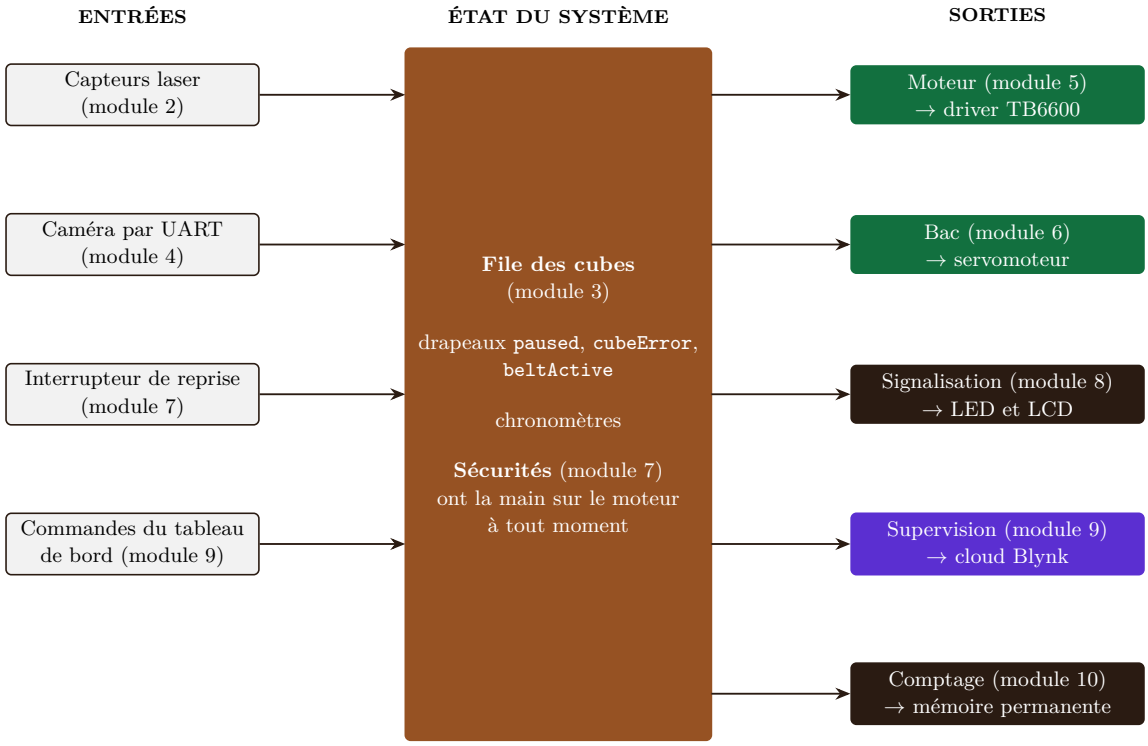


FIGURE 12. Schéma bloc logiciel : entrées, état central, sorties.

4.2 Initialisation et boucle principale

4.2.1 Initialisation : `setup()`

Elle s'exécute une fois, dans un ordre précis. D'abord le port USB (115200 bauds) et le port caméra, puis les sorties de puissance : le driver est activé, le PWM du moteur est configuré puis mis à l'arrêt, le servo est placé en position centrale. Les entrées (capteurs, interrupteur), les LED et le bus I2C suivent, puis l'écran affiche « System ready ». L'état réel des capteurs et de l'interrupteur est relevé *avant* le début de la logique, pour éviter de fausses détections. Les compteurs sont relus depuis la mémoire. Enfin, le Wi-Fi est lancé sans attendre, l'heure réseau

est demandée et la tâche Blynk démarre. Les sorties de puissance sont donc dans un état sûr (moteur arrêté) avant tout traitement.

4.2.2 Boucle principale : `loop()`

La boucle répète indéfiniment la séquence de la tableau 14. Son ordre n'est pas arbitraire : chaque étape suppose que les précédentes ont déjà eu lieu.

Table 14. Ordre des étapes de la boucle principale et justification.

N°	Étape	Pourquoi à cette place
0	Interrupteur, commandes du tableau de bord, ligne d'état, écran	Doit tourner même en pause, sinon on ne pourrait pas reprendre
P	Si PAUSE : jeter les octets, lire les capteurs à blanc, LED, *retour*	La pause neutralise tout le reste de la boucle
1	Lecture des deux capteurs	Un cube détecté crée sa fiche *avant* que sa couleur n'arrive
2	Réception caméra et clavier	La couleur trouve un cube sans couleur à qui s'attribuer ; un `A` peut lever la pause
2b	Si PAUSE (nouveau) : LED, *retour*	Un `A` vient d'arriver : tout s'arrête immédiatement
2c	Si ERREUR : attente du retrait, LED, *retour*	Le tapis est coupé, rien d'autre ne doit avancer
3	Chien de garde	Mesure l'inactivité après avoir pris en compte les signes de vie des étapes 1 et 2
4	Démarrage différé du tapis	Le délai de 300 ms est testé à chaque tour
5	Traitement du capteur de fin	La couleur est connue (étape 2), le tapis tourne (étape 4)
6	Fin du délai de chute du cube de tête	Déclare le cube trié, compte, prépare le cube suivant
7	LED et état envoyé au tableau de bord	Reflète l'état une fois toutes les décisions prises

4.2.3 Deux points de logique à retenir

Détecter avant d'attribuer. Les capteurs sont lus (étape 1) avant la caméra (étape 2) : si un cube vient d'entrer et que sa couleur arrive dans le même tour, la fiche existe déjà et la couleur est correctement attribuée.

Tester la pause deux fois. La pause est testée en tête de boucle, puis de nouveau après la lecture de la caméra : un octet `A` lu à l'étape 2 doit arrêter le tour en cours, sans laisser le démarrage différé ou le traitement du capteur de fin s'exécuter une fois de plus.

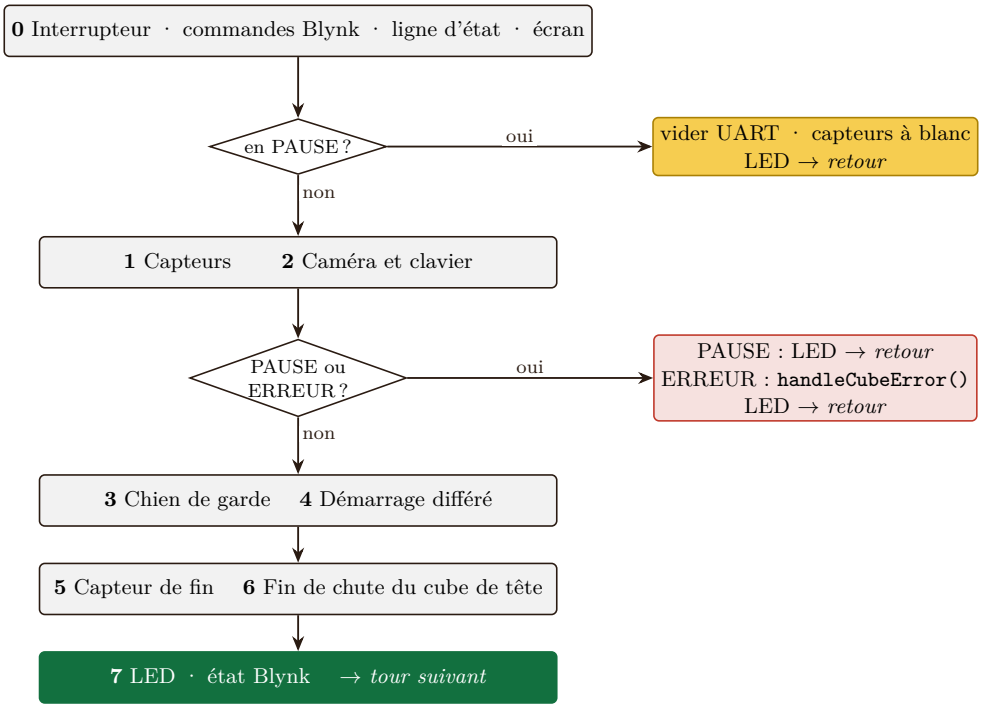


FIGURE 13. Organisation de la boucle principale.

TABLE 15. Affectation des broches de l'ESP32 maître.

Composant	**Broche**	**Signal ou remarque**
Driver TB6600	GPIO5	PUL (impulsions, PWM LEDC canal 0)
	GPIO2	ENA (niveau actif : `LOW`) ; broche de démarrage de l'ESP32
	(non pilotée)	DIR câblée directement à l'alimentation
Servomoteur	GPIO23	Signal PWM 50 Hz, 14 bits (LEDC canal 2)
Capteur laser de début	GPIO33	Entrée analogique (ADC1)
Capteur laser de fin	GPIO32	Entrée analogique (ADC1)
LED verte	GPIO27	Tapis en marche
LED jaune	GPIO26	Tapis arrêté, en pause ou en erreur
LED rouge	GPIO14	Alerte et arrêt d'urgence
Interrupteur de reprise	GPIO12	Entrée avec résistance de rappel interne, vers la masse
Écran LCD (I2C)	GPIO21 / GPIO22	SDA / SCL, adresse `0x27`
Liaison caméra (UART)	GPIO16 / GPIO17	RX (depuis le TX du XIAO) / TX (non utilisée)

4.3 Tableau des broches du microcontrôleur

Deux points de vigilance. La broche GPIO2 est une broche de configuration au démarrage de l'ESP32 (*strapping pin*) : le montage connecté à ENA ne doit pas forcer un niveau incorrect pendant le démarrage. Les entrées analogiques utilisent l'ADC1 car l'ADC2 est inutilisable lorsque le Wi-Fi est actif.

4.4 Tableau des broches virtuelles Blynk

TABLE 16. Broches virtuelles du tableau de bord et règles de traitement.

Pin	Sens	Composant	Contenu ou règle de traitement
V0	↑	Dernier tri	Nom de la dernière couleur triée (YELLOW, GREEN, BLUE, UNKNOWN)
V1	↑	Total tri	Total des cubes triés, toutes couleurs
V2	↑	LED « Run »	255 si le tapis est en marche, sinon 0
V3	↑	LED « Pause »	255 si le tapis n'est pas en marche
V4	↑	LED « Alert »	255 en pause ou en erreur
V5 à V8	↑	Compteurs	Jaune (V5), vert (V6), bleu (V7), inconnu (V8)
V10	↕	Play / Pause	0 : arrêt d'urgence (« DASHBOARD ») ; 1 : reprise si en pause, sinon le bouton est remis sur ON
V11	↑	Graphique	Total triés au cours du temps
V12	↑	Terminal « Logs »	Journal horodaté ; la saisie dans ce terminal est ignorée
V13	↕	« Reset DB »	ON arme la remise à zéro ; repasse sur OFF après chaque tentative
V15	↕	Bac (liste)	Suit le bac réel ; un choix manuel est appliqué s'il est valide et hors pause, sinon la liste est remise sur le bac réel
V16	↕	Code de confirmation	Code de remise à zéro, effacé après chaque saisie

4.5 Scénario complet : le passage d'un cube vert

Avec les valeurs par défaut ($n = 24$ tr/min, $v = 4$ cm/s, $T_{\text{chute}} = 1250$ ms), la tableau 17 suit un cube vert depuis sa détection jusqu'à son comptage. Les instants t_c (réception de la couleur) et t_f (arrivée en fin de tapis) dépendent de la longueur du tapis et de la caméra.

4.6 Plan de test

Le projet se valide par étapes, du composant au système (tableau 18). Chaque programme de test isole une partie pour qu'une panne soit localisée facilement.

L'ordre conseillé est : capteurs (01), moteur (02), Blynk (03), programme complet sans caméra grâce au clavier, puis programme complet avec la caméra. L'écran et le servomoteur n'ont pas de programme dédié : ils sont exercés dès le démarrage du programme principal

TABLE 17. Déroulé d'un cube vert, de la détection au comptage.

Instant	**Événement**	**Effets**
$t = 0$	Front du laser de début	Cube n°1 créé (couleur `?`) ; écran « New cube / On belt : 1 » ; démarrage différé programmé
$t = 300$ ms	Fin du délai de démarrage	Moteur lancé à 80 Hz, tapis à 4 cm/s, LED verte allumée
$t = t_c$	Octet `V` reçu de la caméra	Le cube n°1 devient vert ; le chien de garde est réarmé
$t = t_f$	Front du laser de fin	Arrivée enregistrée (`dropTime`) ; cube de tête de couleur connue : servo à 180° (impulsion 2400 μs, duty 1965) ; V15 positionné sur l'indice 2 ; écran « BIN : / GREEN »
$t = t_f + 1250$ ms	Délai de chute écoulé	Écran « ITEM SORTED / GREEN » ; compteur vert + 1 enregistré en mémoire ; Blynk : V0 = GREEN, V1, V6, V11 et ligne de journal « GREEN sorted | total : n »
Aussitôt après	File vide	Moteur arrêté ; LED jaune ; système en attente d'un nouveau cube

TABLE 18. Programmes de test et facilités de débogage.

Programme	**Objet du test**
`01_laser_ldr_detection`	Lecture de la valeur analogique du capteur laser et affichage sur le moniteur série ; sert à relever les niveaux haut et bas et à choisir le seuil S
`02_stepper_tb6600`	Commande du moteur par le driver TB6600 ; vérifie le câblage PUL, DIR, ENA et la vitesse
`03_blynk_communication`	Échanges avec le tableau de bord, dans les deux sens, avec les mêmes broches virtuelles que le programme principal ; commandes série `J V B X` (tri simulé), `R` (remise à zéro), `S` (état)
`ESP32_master` (simulation clavier)	Tout le tri sans caméra : on tape `J`, `V`, `B`, `X` ou `A` dans le moniteur série
`ESP32_master` (UART brut, ligne d'état)	Chaque octet reçu de la caméra est journalisé ; l'état complet est affiché toutes les 5 s

(message « System ready », servo en position centrale) et lors de la simulation de tri.

4.7 Limites et perspectives

Cette section recense, sans détour, ce que le programme ne fait pas ou ne garantit pas encore.

— **Espacement minimal.** Si deux cubes sont plus proches que D, le bac peut ne pas être orienté à temps ; le programme signale le cas dans le journal mais ne peut pas le corriger.
— **Choix manuel du bac.** Le choix depuis le tableau de bord n'est refusé qu'en pause ; il reste possible pendant la chute d'un cube, ce qui peut le diriger dans un mauvais récipient.
— **Pas de retour de position du servomoteur.** Le programme ne vérifie pas que le bac a atteint sa position.
— **Chien de garde.** Il vide la file, mais les cubes encore présents sur le tapis ne sont alors pas comptés.
— **Mémoire flash.** Quatre écritures par cube trié ; à surveiller pour un usage continu.
— **Secrets dans le code.** Le nom et le mot de passe du réseau Wi-Fi, ainsi que le jeton Blynk, sont écrits en clair dans le fichier source : ils doivent être retirés avant tout partage du code, et le jeton régénéré s'il a déjà circulé.
— **Options de test actives par défaut.** La simulation clavier et le journal brut de l'UART sont activés ; ils sont à désactiver en exploitation.
— **Un seul sens de rotation.** L'entrée DIR du driver est câblée à l'alimentation.

Les perspectives retenues par le club sont : une caméra plus performante avec éclairage contrôlé et un tri par forme avec OpenCV, davantage de couleurs, un tapis plus rapide et plusieurs bacs, et un tableau de bord web hébergé avec base de données dans le cloud (par exemple Firebase) pour conserver un historique long, au-delà de ce que permet Blynk.

PARTIE 5 : CONCLUSION

L'étude a décrit le microprogramme de l'ESP32 maître et les logiques sur lesquelles il repose. La démarche est la même pour chaque module : identifier un problème précis, choisir la technique adaptée, puis vérifier par le calcul qu'elle convient aux valeurs du projet. Cinq idées résument l'ensemble :

— **Comparer des instants plutôt qu'attendre** : tous les délais (démarrage, anti-rebond, clignotement, chute) sont des différences de `millis()`, ce qui laisse la boucle toujours à l'écoute ;
— **Réagir aux fronts, pas aux niveaux** : combinés à l'anti-rebond, ils transforment un signal analogique bruité en événements fiables, à condition que $\Delta t_{\text{deb}} < D/v$;
— **Suivre les cubes dans une file FIFO** : chaque cube garde sa couleur et son état, et le bac suit l'ordre des cubes sans jamais être déplacé pendant une chute ;
— **Isoler le réseau** : la tâche Blynk, sur son propre cœur, communique avec la boucle de tri par une file de messages et des drapeaux, si bien qu'un Wi-Fi défaillant ne peut pas arrêter le tapis ;
— **Mettre la sécurité en premier** : l'arrêt d'urgence est testé avant tout, les chronomètres sont gelés pendant la pause, et une couleur inconnue provoque un arrêt sûr plutôt qu'un mauvais tri.

Cette étude met aussi en évidence des limites assumées (espacement minimal des cubes,

choix manuel du bac non verrouillé, secrets dans le code) listées en partie 4. Les perspectives sont l'amélioration de la vision, l'augmentation de la capacité (couleurs, vitesse, bacs) et un historique de production hébergé au-delà de Blynk.

BIBLIOGRAPHIE

[1] Espressif Systems, *Arduino-ESP32 : documentation* (LEDC, UART, I2C, ADC, Preferences, WiFi). `https://docs.espressif.com/projects/arduino-esp32/`

[2] Espressif Systems, *ESP-IDF Programming Guide* : FreeRTOS, tâches et files de messages. `https://docs.espressif.com/projects/esp-idf/`

[3] FreeRTOS, *FreeRTOS Reference Manual* : tâches, files (*queues*). `https://www.freertos.org/`

[4] Blynk, *Blynk Documentation* : modèles, broches virtuelles, `BLYNK_WRITE`. `https://docs.blynk.io/`

[5] Seeed Studio, *Getting Started with Seeed Studio XIAO ESP32S3 Series*, Seeed Studio Wiki. `https://wiki.seeedstudio.com/xiao_esp32s3_getting_started`

[6] Maker Group Global LLC, *TB6600 Stepper Motor Driver*, V1.0, juillet 2018.

[7] Fabricant non précisé, *SG90 : 9 g Micro Servo*, fiche technique.

[8] Hitachi Semiconductor, *HD44780 : Dot Matrix Liquid Crystal Display Controller/Driver*, fiche technique.

[9] NXP Semiconductors, *UM10204 : I2C-bus specification and user manual*.

[10] Club d'Informatique Industrielle IGE × EPAC, *Étude du système électrique : système automatique de tri par couleur*, 12 octobre 2026.

[11] Club d'Informatique Industrielle IGE × EPAC, *Tri Industriel : tri automatique de cubes par couleur, supervisé à distance en temps réel*, présentation (15 diapositives).

[12] Code source du projet : `ESP32_master.ino`, `01_laser_ldr_detection.ino`, `02_stepper_tb6600.ino`, `03_blynk_communication.ino`.

Illustrations

Les schémas de cette étude ont été réalisés par l'auteur ; le schéma des communications reprend celui de la présentation du projet.

ANNEXE A : INDEX DES FONCTIONS

TABLE 19. Fonctions du programme, module de rattachement et rôle.

Fonction	Module	Rôle
`elapsedSince`	—	Durée écoulée depuis un instant mémorisé
`cubePresent`	2	Cube devant le capteur, d'après la dernière valeur
`risingEdge`	2	Front montant avec anti-rebond
`initSensors`	2	Initialise l'état « précédent » avec l'état réel
`addNewCube`	3	Ajoute un cube à la file, programme le démarrage
`removeCubeAt`	3	Retire un cube à une position donnée
`resetQueue`	3	Vide la file
`handleEndOfBelt`	3	Traite l'arrivée d'un cube au capteur de fin
`advanceToNextCube`	3	Prépare le cube suivant après un tri
`handleCameraUart`	4	Lit tous les octets disponibles
`readCameraByte`	4	Lit un octet et le journalise
`receiveByte`	4	Aiguille un octet : arrêt ou couleur
`receiveColor`	4	Attribue la couleur au plus ancien cube sans couleur
`stepFrequencyHz`	5	Fréquence d'impulsions pour la vitesse courante
`computeDropDelayMs`	5	Délai de chute en millisecondes
`startMotor`, `stopMotor`	5	Lance ou coupe le train d'impulsions
`setRpm`	5	Change la vitesse
`servoAngle`	6	Positionne le servomoteur à un angle
`angleFor`, `menuIndex`	6	Couleur vers angle, couleur vers indice de la liste
`orientBin`	6	Oriente le bac pour une couleur
`colorName`, `isUnknown`	6	Nom d'une couleur, couleur non triable
`enterPause`, `exitPause`	7	Arrêt d'urgence et reprise avec gel des chronomètres
`handleToggleSwitch`	7	Lecture et anti-rebond de l'interrupteur
`triggerCubeError`	7	Arrêt sur un cube inconnu
`handleCubeError`	7	Attente du retrait, reprise automatique
`signalActivity`	7	Réarme le chien de garde
`updateLeds`, `updateAlert`	8	Commande des LED et clignotement
`triggerAlertX`	8	Lance les trois clignotements d'un `X`
`writeLcd`, `showLcd`, `refreshLcd`	8	Affichage et réécriture périodique de l'écran
`printHeartbeat`	8	Ligne d'état périodique (débogage)
`blynkTask`	9	Tâche réseau du cœur 0
`pushInt`, `pushTxt`, `termLog`	9	Dépôt de messages dans la file
`timestamp`	9	Heure réseau ou temps depuis le démarrage
`handleBlynkCommands`	9	Traite les commandes du tableau de bord
`sendFullState`, `syncBlynkState`	9	Resynchronisation complète, envoi sur changement
`totalSorted`, `saveCounters`, `pushCounters`	10	Total, sauvegarde en flash, envoi au tableau de bord

Fonction	Module	Rôle
`recordSort`, `resetCounters`	10	Comptage d'un cube trié, remise à zéro
`setup`, `loop`	4.2	Initialisation et boucle principale